

Introduction to Databases

Lecture 4: Transactions and Reliability

The ACID Properties

A transaction is a sequence of database operations that is treated as a single logical unit of work. Relational databases guarantee four properties for every transaction, known by the acronym ACID.

- **Atomicity** - a transaction either completes entirely or has no effect at all; there is no partial completion.
- **Consistency** - a transaction can only bring the database from one valid state to another, never violating defined rules such as constraints.
- **Isolation** - concurrent transactions produce the same result as if they had run one after another, sequentially.
- **Durability** - once a transaction is committed, it remains committed even in the event of a power loss or crash.

Transaction Isolation Levels

The SQL standard defines four isolation levels, each permitting a different set of concurrency anomalies in exchange for better performance.

- **Read Uncommitted** - transactions can see uncommitted changes made by other transactions (dirty reads). The weakest and fastest level.
- **Read Committed** - a transaction only ever sees data that has been committed. This is the default isolation level in PostgreSQL.
- **Repeatable Read** - if a transaction reads a row twice, it will see the same data both times, preventing non-repeatable reads.
- **Serializable** - the strongest level; concurrent transactions behave as if they had executed one at a time, in some serial order.

Higher isolation levels reduce anomalies but increase the chance of transactions blocking each other or being forced to retry after a conflict.

Indexing: B-Trees and Hash Indexes

An index is a separate data structure that lets the database find rows without scanning the entire table. Two common index types are B-tree and hash indexes.

- **B-tree indexes** keep keys in sorted order, which supports equality lookups, range queries (e.g. age BETWEEN 20 AND 30), and ORDER BY. This is the default index type in PostgreSQL and MySQL.
- **Hash indexes** store a hash of each key and support only equality lookups (e.g. WHERE id = 5); they cannot answer range queries because hashing destroys the original ordering of the values.

Rule of thumb: use a B-tree index unless you know every query against that column will be an exact-match lookup and you need marginally faster point lookups than a B-tree provides.

Normal Forms

Normalization is the process of structuring tables to reduce data redundancy and avoid update anomalies.

- **First Normal Form (1NF)** - every column holds a single, atomic value; no repeating groups or arrays inside a single column.
- **Second Normal Form (2NF)** - the table is in 1NF, and every non-key column depends on the whole primary key, not just part of it (relevant mainly for composite keys).
- **Third Normal Form (3NF)** - the table is in 2NF, and every non-key column depends only on the primary key, not on another non-key column (no transitive dependencies).

Example: storing both a customer's city and that city's country in an orders table violates 3NF, because country depends on city, not on the order itself.

The CAP Theorem

The CAP theorem states that a distributed data store can provide at most two of the following three guarantees at the same time, during a network partition:

- **Consistency** - every read receives the most recent write or an error.
- **Availability** - every request receives a non-error response, without the guarantee that it contains the most recent write.
- **Partition tolerance** - the system continues to operate despite an arbitrary number of messages being dropped or delayed between nodes.

Because real networks do experience partitions, the practical choice is between CP (consistent but may reject requests during a partition) and AP (available but may serve stale data during a partition). A single-node relational database sidesteps this trade-off entirely, since there is no partition to tolerate.